

AI Destekli Mini Embedded Linux Sistemi Geliřtirme Projesi

Marmara Üniversitesi - Gömülü Sistemler

Öğretim Üyesi Dr. Sebahattin Babur

170423053 - Mehmet İhsan Ekinçi

Haziran 2026

GitHub: <https://github.com/mihsanekinci/ai-driven-embedded-linux>

1) GİRİŞ

1.1 Projenin Amacı

Bu projede yapay zekâ destekli geliştirme süreci kullanılarak uçtan uca çalışan bir Embedded Linux sistemi geliştirilmiştir. Proje kapsamında ARM mimarisinde çalışan bir C uygulaması cross-compile edilmiş, BusyBox tabanlı minimal bir RootFS oluşturulmuş, sistem QEMU üzerinde boot edilmiş, fork/pipe ile IPC örneği geliştirilmiş, Device Tree düzenlenmiş ve character driver analizi yapılmıştır.

Projenin temel amacı "AI ile kod üretmek" değil, "AI destekli mühendislik geliştirme süreci" uygulamaktır. Üretilen her kod parçası analiz edilmiş, hatalar tespit edilip çözülmüş ve tüm sistem çalışır hale getirilmiştir.

1.2 Kullanılan Araçlar

Claude AI — kod üretimi, hata ayıklama ve analiz

arm-linux-gnueabi-gcc 15.2.0 — ARM cross-compiler

QEMU 10.2.1 — ARM sistem emülatörü

BusyBox — minimal RootFS oluşturma

dtc 1.7.2 — Device Tree Compiler

VS Code + Claude Code eklentisi — geliştirme ortamı

1.3 Geliştirme Ortamı

Host makine: Ubuntu VM (VMware Workstation)

Hedef mimari: ARM Cortex-A9 (vexpress-a9 board)

Kernel: Debian 6.1.0-49 ARM (pre-built)

Proje dizini: ~/gomulu_sistemler/ai-driven-embedded-linux

2) AŞAMA 1 — CROSS COMPILATION

2.1 Kullanılan Prompt

Projenin tamamını kapsayan aşağıdaki prompt verildi:

"6 aşamalı bir Embedded Linux projesi üzerinde çalışıyorum: 1. ARM için C uygulaması cross-compile etmek, 2. BusyBox tabanlı minimal RootFS oluşturmak, 3. QEMU ARM üzerinde Linux kernel boot etmek, 4. fork() ve pipe() ile IPC uygulaması geliştirmek, 5. Bir sensör için Device Tree node yazmak, 6. Character driver analizi yapmak. Ortamım: Ubuntu, arm-linux-gnueabi-gcc 15.2.0, QEMU 10.2.1, dtc 1.7.2 kurulu. Aşama 1 ile başlayalım. Her

saniye sahte sıcaklık verisi üreten ve terminale yazdıran bir C uygulaması yaz. Ardından ARM için cross-compile komutunu ver."

Bu prompt tercih edildi çünkü tek seferde tüm proje bağlamı verildi. Böylece AI, uygulamayı baştan ARM uyumlu yazdı ve sonraki aşamalarla tutarlı bir yapı kuruldu.

2.2 Kodun Açıklaması

temperature_sensor.c, stdio.h, stdlib.h, time.h ve unistd.h standart C kütüphanelerini kullanarak her saniye sahte sıcaklık değeri üretip terminale yazdıran minimal bir sensör simülatörüdür.

Sıcaklık değerleri, 20.0°C taban değerine $\text{rand()} / \text{RAND_MAX} * 15.0$ formülüyle eklenen rastgele bir bileşenle 20–35°C aralığında üretilir. `srand(time(NULL))` çağrısı her çalıştırmada farklı bir dizi sağlar. `time()` ve `localtime()` ile alınan sistem saati [SS:DD:SS] formatında zaman damgası olarak çıktıya eklenir, `fflush(stdout)` ile tamponlama devre dışı bırakılır ve `while(1)` döngüsü `sleep(1)` ile her saniye bir ölçüm üretecek şekilde sonsuza dek çalışır.

2.3 Derleme Komutları

Kullanılan derleme komutu:

```
arm-linux-gnueabi-gcc -o temperature_sensor_arm temperature_sensor.c -static -O2
```

Komut açıklaması:

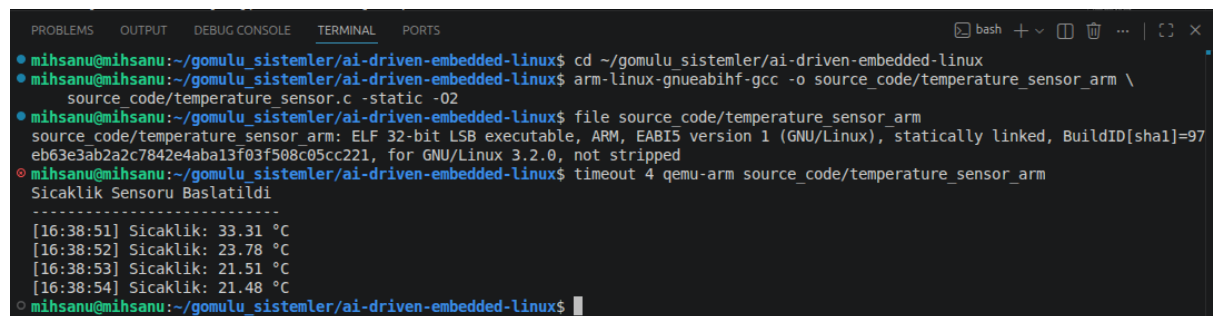
arm-linux-gnueabi-gcc: ARM hard-float ABI cross-compiler. Ubuntu'da çalışır, ARM için binary üretir.

-static: dinamik kütüphane bağımlılığı olmadan derleme yapar. Minimal rootfs'te libc.so bulunmadığı için bu parametre zorunludur.

-O2: boyut ve hız optimizasyonu sağlar.

Çıktı: ARM ELF32 statik binary (550KB)

2.4 Ekran Görüntüsü



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ cd ~/gomulu_sistemler/ai-driven-embedded-linux
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ arm-linux-gnueabi-gcc -o source_code/temperature_sensor_arm \
source_code/temperature_sensor.c -static -O2
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ file source_code/temperature_sensor_arm
source_code/temperature_sensor_arm: ELF 32-bit LSB executable, ARM, EABI5 version 1 (GNU/Linux), statically linked, BuildID[sha1]=97
eb63e3ab2a2c7842e4ab13f03f508c05cc221, for GNU/Linux 3.2.0, not stripped
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ timeout 4 qemu-arm source_code/temperature_sensor_arm
Sicaklik Sensoru Baslatildi
-----
[16:38:51] Sicaklik: 33.31 °C
[16:38:52] Sicaklik: 23.78 °C
[16:38:53] Sicaklik: 21.51 °C
[16:38:54] Sicaklik: 21.48 °C
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$
```

3) AŞAMA 2 — ROOTFS OLUŞTURMA

3.1 Kullanılan Prompt

İlk aşamanın ardından "Aşama 2 için devam edelim" komutu verildi. AI önceki bağlamı koruyarak BusyBox tabanlı minimal RootFS oluşturma adımlarını verdi. Bağlamın korunması sayesinde ayrı bir açıklama yapmaya gerek kalmadı; AI projenin ARM hedefli olduğunu bilerek adımları ona göre şekillendirdi.

3.2 Dizin Yapısı Açıklaması

Oluşturulan rootfs/ dizin yapısı aşağıdaki gibidir:

rootfs/ └── bin/ → BusyBox binary ve sembolik bağlantılar (ls, cp, grep, vi...) └── sbin/ → Sistem komutları (init, mount, reboot, insmod...) └── usr/bin/ → temperature_sensor ARM binary └── etc/ └── inittab → Boot sırası: sysinit → rcS → ash shell └── passwd, group, fstab └── init.d/rcS → proc/sys/tmp/run mount, hostname ayarı └── lib/ → Statik derleme nedeniyle büyük ölçüde boş └── dev/ → Çalışma zamanında mdev tarafından doldurulur └── proc/ → Çekirdek sanal dosya sistemi mount noktası └── sys/ → sysfs mount noktası └── tmp/, run/, home/, root/, mnt/, opt/, var/log/

bin/busybox tek bir binary'dir. ls, sh, mount, grep gibi yaklaşık 401 komut aynı binary'ye sembolik bağlantıyla yönlendirilir. Bu yapı sayesinde tam bir komut seti yalnızca birkaç MB'lık tek dosyayla sağlanır.

sbin/init, BusyBox'ın init programıdır. /etc/inittab dosyasını okuyarak boot sürecini yönetir, rcS scriptini çalıştırır ve ardından ash shell başlatır.

3.3 BusyBox Derleme ve Hata Ayıklama

BusyBox kaynak koddan ARM için derlenirken iki kritik sorunla karşılaşıldı.

Hata 1 — tc.c Derleme Hatası:

BusyBox derlenirken tc.c dosyasında derleme hatası alındı. Sorun, tc komutunun derleme ortamıyla uyumsuz olmasından kaynaklanıyordu. Çözüm olarak make menuconfig arayüzü açılarak tc komutu devre dışı bırakıldı ve derleme başarıyla tamamlandı.

Hata 2 — ENOEXEC (error -8):

initramfs içine eklenen BusyBox binary'si host makineden kopyalandığı için x86-64 formatındaydı. ARM kernel bu binary'yi çalıştıramadı ve şu hatayı verdi:

Failed to execute /sbin/init (error -8: ENOEXEC)

Tanı için file bin/busybox komutu çalıştırıldı ve "ELF 64-bit LSB executable, x86-64" çıktısı alındı. Çözüm olarak Debian armhf reposundan ARM statik BusyBox binary'si indirildi ve initramfs yeniden paketlenildi.

3.4 initramfs Oluşturma

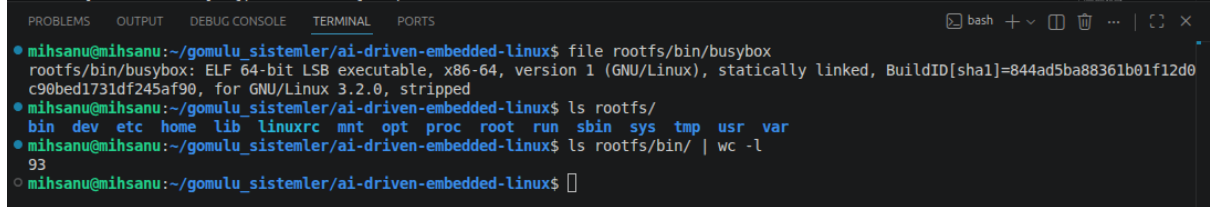
Kullanılan komut:

```
cd rootfs && find . | cpio -o -H newc | gzip > ../qemu/initramfs.cpio.gz
```

ext2 disk image yerine initramfs tercih edilmesinin nedenleri:

initramfs boyutu 1.2MB olup GitHub'a push edilebilir. ext2 image ise 64MB olup GitHub limitini aşmaktadır. Ayrıca QEMU'da kullanımı -initrd parametresiyle çok daha basittir, loop mount gerektirmez.

3.5 Ekran Görüntüsü



```
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ file rootfs/bin/busybox
rootfs/bin/busybox: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically linked, BuildID[sha1]=844ad5ba88361b01f12d0c90bed1731df245af90, for GNU/Linux 3.2.0, stripped
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ ls rootfs/
bin dev etc home lib linuxrc mnt opt proc root run sbin sys tmp usr var
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ ls rootfs/bin/ | wc -l
93
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$
```

4) AŞAMA 3 — QEMU BOOT

4.1 Kullanılan Prompt

Aşama 3 için aşağıdaki prompt Claude Code'a verildi:

"Görev: ARM Linux kernel indir ve QEMU ile initramfs boot et. Ortam: Proje dizini ~/gomulu_sistemler/ai-driven-embedded-linux, initramfs qemu/initramfs.cpio.gz mevcut (1.6MB, BusyBox tabanlı ARM), Cross-compiler arm-linux-gnueabi-hf-gcc (32-bit ARM hard-float), QEMU qemu-system-arm 10.2.1, Host Ubuntu VM. Makine tipi: qemu-system-arm -M vexpress-a9, Konsol: ttyAMA0, DTB: vexpress-v2p-ca9.dtb. Kernel derleme saatlerce süreceğinden Debian paketinden hazır zImage indirilmesi istendi. QEMU boot scripti yazılması, boot logunun kaydedilmesi ve ilk 50 satırın screenshots klasörüne kopyalanması talep edildi."

Prompt bu şekilde tasarlandı çünkü kernel sıfırdan derleme 2-3 saat sürmektedir. Hazır Debian ARM binary'si kullanmak hem zaman kazandırdı hem de kararlı, test edilmiş bir kernel sağladı. Mevcut dosyalar (initramfs, cross-compiled binary) temel alınarak prompt oluşturuldu.

4.2 Boot Komutu Açıklaması

Oluşturulan qemu/run_qemu.sh içeriği:

```
qemu-system-arm -M vexpress-a9 -kernel "PROJ/qemu/zImage"-dtb"PROJ/qemu/zImage" -dtb " PROJ/qemu/zImage"-dtb"PROJ/qemu/vexpress-v2p-ca9.dtb" -initrd "$PROJ/qemu/initramfs.cpio.gz" -append "console=ttyAMA0 rdinit=/sbin/init" -serial stdio -display none -m 256
```

Parametre açıklamaları:

-M vexpress-a9: ARM Versatile Express Cortex-A9 board emülasyonu yapar. Gerçek bir donanım yerine yazılımla taklit edilen bir board oluşturur.

-kernel zImage: Sıkıştırılmış ARM Linux kernel binary dosyasıdır (5.2MB). Sistemin çekirdeğidir.

-dtb vexpress-v2p-ca9.dtb: Donanım tanım dosyasıdır. Kernel'e hangi donanımların mevcut olduğunu bildirir.

-initrd initramfs.cpio.gz: BusyBox tabanlı minimal rootfs arşividir. Boot sırasında RAM'e açılır.

console=ttyAMA0: Kernel mesajlarını ilk UART seri portuna yönlendirir.

rdinit=/sbin/init: initramfs içinde çalıştırılacak ilk süreçtir.

-serial stdio: Seri konsolu terminale bağlar, çıktı yakalamayı sağlar.

-display none: Grafik penceresi açılmaz, headless modda çalışır.

-m 256: Emülasyona 256MB RAM tahsis edilir.

Not: -nographic yerine -serial stdio -display none kullanıldı. -nographic TTY gerektirdiğinden pipe ve script ortamında çıktı üretilmiyordu.

4.3 Hata Ayıklama

Hata — ENOEXEC (error -8):

Belirti: Kernel başarıyla boot oluyordu ancak init süreci başlamıyordu. Kernel şu mesajı verdi: "Failed to execute /sbin/init (error -8: ENOEXEC)"

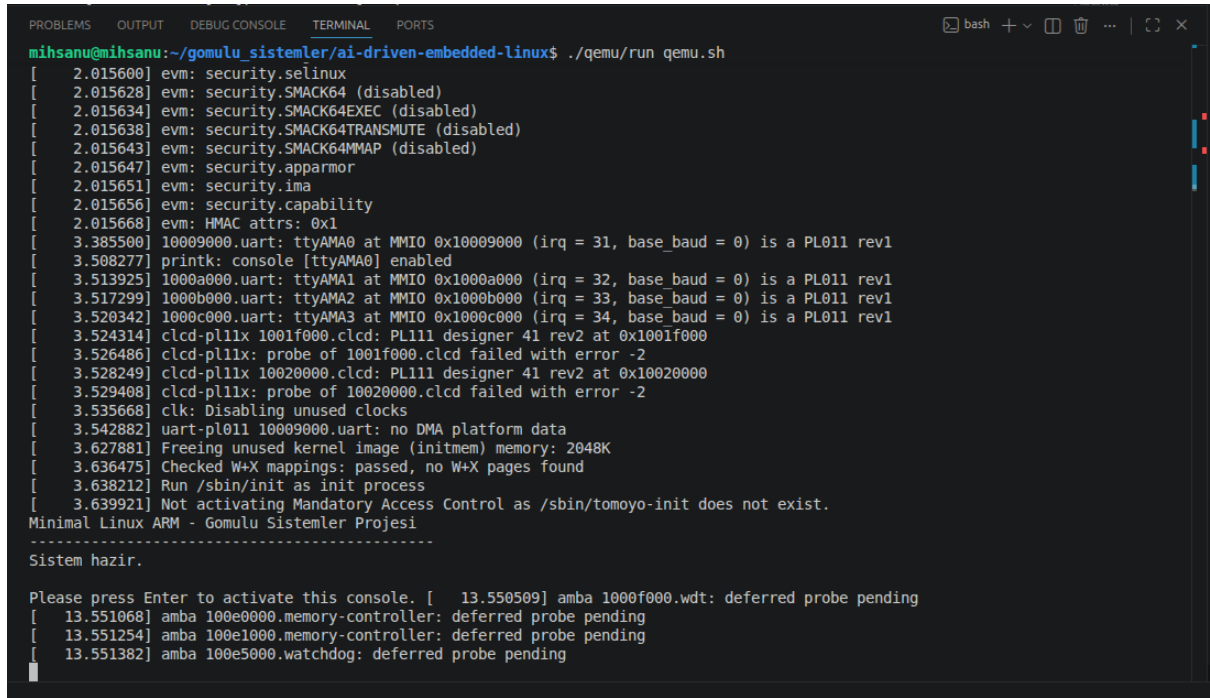
Sebebi Analizi: ENOEXEC, Execute Format Error anlamına gelir. Kernel, initramfs içindeki BusyBox binary'sini çalıştırmaya çalıştı ancak binary x86-64 formatındaydı. ARM mimarisindeki kernel farklı formattaki binary'yi execute edemez. Binary host makineden kopyalandığı için farkında olmadan x86 formatında sisteme dahil edilmişti.

Tanı: file bin/busybox komutu çalıştırıldı. Çıktı "ELF 64-bit LSB executable, x86-64" şeklindeydi. ARM binary olması gerekirken x86-64 olduğu bu şekilde tespit edildi.

Çözüm: Debian armhf reposundan statik derlenmiş ARM BusyBox binary'si indirildi ve initramfs yeniden paketlenildi.

Başarı Kanıtı: "Please press Enter to activate this console"

4.4 Ekran Görüntüsü



```
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ ./qemu/run qemu.sh
[ 2.015600] evm: security.selinux
[ 2.015628] evm: security.SMACK64 (disabled)
[ 2.015634] evm: security.SMACK64EXEC (disabled)
[ 2.015638] evm: security.SMACK64TRANSMUTE (disabled)
[ 2.015643] evm: security.SMACK64MMAP (disabled)
[ 2.015647] evm: security.apparmor
[ 2.015651] evm: security.ima
[ 2.015656] evm: security.capability
[ 2.015668] evm: HMAC attrs: 0x1
[ 3.385500] 10009000.uart: ttyAMA0 at MMIO 0x10009000 (irq = 31, base_baud = 0) is a PL011 rev1
[ 3.508277] printk: console [ttyAMA0] enabled
[ 3.513925] 1000a000.uart: ttyAMA1 at MMIO 0x1000a000 (irq = 32, base_baud = 0) is a PL011 rev1
[ 3.517299] 1000b000.uart: ttyAMA2 at MMIO 0x1000b000 (irq = 33, base_baud = 0) is a PL011 rev1
[ 3.520342] 1000c000.uart: ttyAMA3 at MMIO 0x1000c000 (irq = 34, base_baud = 0) is a PL011 rev1
[ 3.524314] clcd-pll1x 1001f000.clcd: PL111 designer 41 rev2 at 0x1001f000
[ 3.526486] clcd-pll1x: probe of 1001f000.clcd failed with error -2
[ 3.528249] clcd-pll1x 10020000.clcd: PL111 designer 41 rev2 at 0x10020000
[ 3.529408] clcd-pll1x: probe of 10020000.clcd failed with error -2
[ 3.535668] clk: Disabling unused clocks
[ 3.542882] uart-pl011 10009000.uart: no DMA platform data
[ 3.627881] Freeing unused kernel image (initmem) memory: 2048K
[ 3.636475] Checked W+X mappings: passed, no W+X pages found
[ 3.638212] Run /sbin/init as init process
[ 3.639921] Not activating Mandatory Access Control as /sbin/tomoyo-init does not exist.
Minimal Linux ARM - Gomulu Sistemler Projesi
-----
Sistem hazır.

Please press Enter to activate this console. [ 13.550509] amba 1000f000.wdt: deferred probe pending
[ 13.551068] amba 100e0000.memory-controller: deferred probe pending
[ 13.551254] amba 100e1000.memory-controller: deferred probe pending
[ 13.551382] amba 100e5000.watchdog: deferred probe pending
```

5) AŞAMA 4 — IPC SİSTEMİ

5.1 Kullanılan Prompt

Aşama 3 tamamlandıktan sonra Claude Code'un önerdiği plan onaylandı: "Aşama 4 — IPC: fork() ve pipe() kullanan bir demo yaz. Parent process 5 sıcaklık ölçümü üretip pipe'a yazsın, child process pipe'tan okuyup ekrana yazdırsın. ARM için statik derle, QEMU'da çalıştır."

Bu planı onaylamak için "Evet, devam edelim" promptu verildi.

5.2 fork() + pipe() Açıklaması

ipc_demo.c, pipe() sistem çağrısı ile tek yönlü bir IPC kanalı kurar. pipefd[0] okuma ucu, pipefd[1] ise yazma ucudur. fork() çağrısı bu pipe zaten açıkken yapıldığından her iki süreç de her iki ucu miras alır.

Süreç ayrışması fork() sonrası şu şekildedir:

pipe() oluşturulur → pipefd[0] okuma, pipefd[1] yazma ucu olur. fork() çağrılır. Child süreç (pid==0) yazma ucunu kapatır ve yalnızca okur. Parent süreç (pid>0) okuma ucunu kapatır ve yalnızca yazar.

Kullanılmayan uçları kapatmak zorunludur. Parent pipefd[1]'i kapatmadan EOF sinyali iletilemez. Child pipefd[1]'i kapatmazsa read() asla dönmez.

Parent davranışı: MSG_COUNT=5 kez döngü çalışır. Her turda fake_temperature() ile sıcaklık üretilir, snprintf ile formatlanır ve write(pipefd[1]) ile pipe'a iletilir. Turlar arası sleep(1) ile 1 saniyelik gecikme eklenir.

Child davranışı: read(pipefd[0]) çağrısı bloklayarak bekler. Her gelen mesajı printf ile yazdırır. Parent pipe'ı kapattığında EOF üretilir, read() sıfır döner ve child döngüden çıkarak sonlanır.

Senkronizasyon: Parent, waitpid() ile child'ın bitmesini bekledikten sonra "Tamamlandı" mesajını basar. Bu sayede çıktılar karışmaz.

5.3 Hata Ayıklama

Hata ayıklama süreci sorunsuz tamamlandı. Üç önemli teknik gözlem yapıldı:

Program QEMU'da PID=1 ile çalıştı. Bu, sistemdeki tek process olduğunun göstergesidir ve minimal gömülü Linux yapısını doğrular.

Binary statik derlendi (486KB). Minimal rootfs'te dinamik kütüphane bulunmadığından -static parametresi zorunluydu.

fork() ve pipe() ARM mimarisinde x86 ile özdeş davranış gösterdi. Bu, Linux sistem çağrılarının mimari bağımsızlığını kanıtlar.

5.4 Ekran Görüntüsü

```
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ ./qemu/run qemu.sh
[ 3.105264] uart-pl011 10009000.uart: no DMA platform data
[ 3.187733] Freeing unused kernel image (initmem) memory: 2048K
[ 3.197028] Checked W+X mappings: passed, no W+X pages found
[ 3.198922] Run /sbin/init as init process
[ 3.200635] Not activating Mandatory Access Control as /sbin/tomoyo-init does not exist.
Minimal Linux ARM - Gomulu Sistemler Projesi
-----
Sistem hazır.

Please press Enter to activate this console.

BusyBox v1.35.0 (Debian 1:1.35.0-4+deb12u1+b1) built-in shell (ash)
Enter 'help' for a list of built-in commands.

/ # [ 13.291029] amba 1000f000.wdt: deferred probe pending
[ 13.291206] amba 100e0000.memory-controller: deferred probe pending
[ 13.291348] amba 100e1000.memory-controller: deferred probe pending
[ 13.291467] amba 100e5000.watchdog: deferred probe pending

/ # /usr/bin/ipc demo arm
IPC Demo - fork() + pipe()
-----
[Parent PID=95] Veri gönderiliyor...
[Child PID=96] Okuma bekleniyor...
[Child PID=96] Alindi: Olcum #1: 20.0 C
[Child PID=96] Alindi: Olcum #2: 23.0 C
[Child PID=96] Alindi: Olcum #3: 26.0 C
[Child PID=96] Alindi: Olcum #4: 29.0 C
[Child PID=96] Alindi: Olcum #5: 32.0 C
[Child PID=96] Pipe kapandi, cikiliyor.
[Parent PID=95] Tamamlandı.
/ #
```


6) AŞAMA 5 — DEVICE TREE

6.1 Kullanılan Prompt

Aşama 5 için aşağıdaki prompt Claude Code'a verildi:

"Görev: vexpress-v2p-ca9.dtb dosyasını analiz et, temperature-sensor node ekle, derle ve QEMU'da doğrula. Önce dtc -I dtb komutuyla mevcut DTB tersine çevrilecek, yapı incelenecek. Ardından i2c node varsa o node altına, yoksa root altına platform device olarak temperature-sensor node eklenecek. Yeni DTS derlenecek, QEMU'da modified DTB ile boot edilecek ve /proc/device-tree altında node'un görüldüğü doğrulanacak."

Bu prompt tercih edildi çünkü sadece node eklemek yerine önce mevcut yapıyı anlamak daha doğru bir mühendislik yaklaşımıdır. Tersine mühendislik (decompile) adımı sayesinde node'un nereye ekleneceği analiz edilerek karar verildi.

6.2 DTS Analizi

vexpress-v2p-ca9.dtb dosyası dtc -I dtb -O dts komutuyla tersine çevrilerek 723 satırlık vexpress-original.dts dosyası elde edildi. Analiz bulguları şu şekildedir:

CPU: 4x ARM Cortex-A9 (cpu@0 ile cpu@3 arası node'lar)

Bellek: memory@60000000, boyut 0x40000000 (1GB RAM)

Konsol UART: uart@9000, ttyAMA0 olarak tanımlı (PL011 UART)

I2C birincil bus: i2c@16000, compatible değeri "arm,versatile-i2c", alias: i2c0

I2C ikincil bus: i2c@2000, alias: i2c1

i2c@16000 node'u seçildi çünkü sistemin birincil I2C veri yoludur ve i2c0 alias'ına sahiptir. Bu node içinde zaten dvi-transmitter@39 ve dvi-transmitter@60 node'ları bulunmaktaydı. Bu durum, sensör eklemenin doğru konuma yapıldığını doğrulamak için referans noktası işlevi gördü.

6.3 Eklenen Node Açıklaması

i2c@16000 node'u altına aşağıdaki sensör node'u eklendi:

```
temperature_sensor: temp-sensor@48 { compatible = "national,lm75"; reg = <0x48>; status = "okay"; label = "CPU Temperature Sensor"; };
```

Derleme komutu: dtc -I dts -O dtb -o dts/vexpress-modified.dtb dts/vexpress-modified.dts

Özellik açıklamaları:

compatible = "national,lm75": Kernel'in hangi sürücüyü bu node ile eşleştireceğini belirler. Bu değer, sürücü ile DTS arasındaki sözleşmedir.

reg = 0x48: I2C bus üzerindeki slave adresidir. 0x48, LM75 sıcaklık sensörünün varsayılan I2C adresidir.

status = "okay": Node'un aktif olduğunu bildirir. "disabled" yazılıysaydı kernel bu cihazı yok sayardı.

6.4 Doğrulama

QEMU vexpress-modified.dtb ile boot edildikten sonra /proc/device-tree üzerinden node varlığı doğrulandı. Çıktı şu şekildedir:

/proc/device-tree/.../iofpga@7,00000000/i2c@16000/ dizininde şu node'lar görüldü: dvi-transmitter@39, dvi-transmitter@60, temp-sensor@48

cat temp-sensor@48/compatible komutu çıktısı: national,lm75

Sonuç: temp-sensor@48 node'u /proc/device-tree içinde başarıyla görüldü ve compatible değeri doğrulandı.

6.5 Ekran Görüntüsü

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ cd ~/gomulu_sistemler/ai-driven-embedded-linux
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ dtc -I dtb -O dts -o /tmp/check.dts qemu/vexpress-v2p-ca9.dtb
/tmp/check.dts: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/flash@0,00000000: simple-bus unit address format error, expected "0"
/tmp/check.dts: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/psram@2,00000000: simple-bus unit address format error, expected "20000000"
/tmp/check.dts: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/ethernet@3,02000000: simple-bus unit address format error, expected "30200000"
/tmp/check.dts: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/usb@3,03000000: simple-bus unit address format error, expected "30300000"
/tmp/check.dts: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/iofpga@7,00000000: simple-bus unit address format error, expected "70000000"
/tmp/check.dts: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/fixed-regulator-0: missing or empty reg/ranges property
/tmp/check.dts: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/clk24mhz: missing or empty reg/ranges property
/tmp/check.dts: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/refclk1mhz: missing or empty reg/ranges property
/tmp/check.dts: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/refclk32khz: missing or empty reg/ranges property
/tmp/check.dts: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/leds: missing or empty reg/ranges property
/tmp/check.dts: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/mcc: missing or empty reg/ranges property
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ wc -l /tmp/check.dts
723 /tmp/check.dts
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ grep -A 6 "temp-sensor" dts/vexpress-modified.dts
        temperature sensor: temp-sensor@48 {
            compatible = "national,lm75";
            reg = <0x48>;
            status = "okay";
            label = "CPU Temperature Sensor";
        };
};
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ dtc -I dts -O dtb -o dts/vexpress-modified.dtb dts/vexpress-modified.dts
s && echo "DTB derlendi OK"
dts/vexpress-modified.dts:29.21-37.6: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/flash@0,00000000: simple-bus unit address format error, expected "0"
dts/vexpress-modified.dts:39.21-43.6: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/psram@2,00000000: simple-bus unit address format error, expected "20000000"
dts/vexpress-modified.dts:45.24-55.6: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/ethernet@3,02000000: simple-bus unit address format error, expected "30200000"
dts/vexpress-modified.dts:57.19-62.6: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/usb@3,03000000: simple-bus unit address format error, expected "30300000"
dts/vexpress-modified.dts:64.22-298.6: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/iofpga@7,00000000: simple-bus unit address format error, expected "70000000"
dts/vexpress-modified.dts:300.22-307.6: Warning (simple_bus_reg): /bus@40000000/motherboard-bus@40000000/fixed-regulator-0: missing or empty reg/ranges property
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ dtc -I dts -O dtb -o dts/vexpress-modified.dtb dts/vexpress-modified.dts && echo "DTB derlendi OK"
clocks: cell 0 is not a phandle reference
dts/vexpress-modified.dts:214.6-42: Warning (clocks_property): /bus@40000000/motherboard-bus@40000000/iofpga@7,00000000/timer@12000:
clocks: cell 2 is not a phandle reference
dts/vexpress-modified.dts:214.6-42: Warning (clocks_property): /bus@40000000/motherboard-bus@40000000/iofpga@7,00000000/timer@12000:
clocks: cell 4 is not a phandle reference
dts/vexpress-modified.dts:269.6-22: Warning (clocks_property): /bus@40000000/motherboard-bus@40000000/iofpga@7,00000000/rtc@17000:cl
ocks: cell 0 is not a phandle reference
dts/vexpress-modified.dts:284.6-27: Warning (clocks_property): /bus@40000000/motherboard-bus@40000000/iofpga@7,00000000/clcd@1f000:c
locks: cell 0 is not a phandle reference
dts/vexpress-modified.dts:284.6-27: Warning (clocks_property): /bus@40000000/motherboard-bus@40000000/iofpga@7,00000000/clcd@1f000:c
locks: cell 1 is not a phandle reference
dts/vexpress-modified.dts:529.3-24: Warning (clocks_property): /clcd@10020000:clocks: cell 0 is not a phandle reference
dts/vexpress-modified.dts:529.3-24: Warning (clocks_property): /clcd@10020000:clocks: cell 1 is not a phandle reference
dts/vexpress-modified.dts:546.3-19: Warning (clocks_property): /memory-controller@100e0000:clocks: cell 0 is not a phandle reference
dts/vexpress-modified.dts:554.3-19: Warning (clocks_property): /memory-controller@100e1000:clocks: cell 0 is not a phandle reference
dts/vexpress-modified.dts:562.3-29: Warning (clocks_property): /timer@100e4000:clocks: cell 0 is not a phandle reference
dts/vexpress-modified.dts:562.3-29: Warning (clocks_property): /timer@100e4000:clocks: cell 1 is not a phandle reference
dts/vexpress-modified.dts:562.3-29: Warning (clocks_property): /timer@100e4000:clocks: cell 2 is not a phandle reference
dts/vexpress-modified.dts:571.3-24: Warning (clocks_property): /watchdog@100e5000:clocks: cell 0 is not a phandle reference
dts/vexpress-modified.dts:571.3-24: Warning (clocks_property): /watchdog@100e5000:clocks: cell 1 is not a phandle reference
dts/vexpress-modified.dts:138.6-34: Warning (gpio_property): /bus@40000000/motherboard-bus@40000000/iofpga@7,00000000/mmc@5000:cd-
gpio: cell 0 is not a phandle reference
dts/vexpress-modified.dts:139.6-34: Warning (gpio_property): /bus@40000000/motherboard-bus@40000000/iofpga@7,00000000/mmc@5000:wp-
gpio: cell 0 is not a phandle reference
dts/vexpress-modified.dts:338.6-31: Warning (gpio_property): /bus@40000000/motherboard-bus@40000000/leds/user1:gpio: cell 0 is not
a phandle reference
dts/vexpress-modified.dts:344.6-31: Warning (gpio_property): /bus@40000000/motherboard-bus@40000000/leds/user2:gpio: cell 0 is not
a phandle reference
dts/vexpress-modified.dts:350.6-31: Warning (gpio_property): /bus@40000000/motherboard-bus@40000000/leds/user3:gpio: cell 0 is not
a phandle reference
dts/vexpress-modified.dts:356.6-31: Warning (gpio_property): /bus@40000000/motherboard-bus@40000000/leds/user4:gpio: cell 0 is not
a phandle reference
dts/vexpress-modified.dts:362.6-31: Warning (gpio_property): /bus@40000000/motherboard-bus@40000000/leds/user5:gpio: cell 0 is not
a phandle reference
dts/vexpress-modified.dts:368.6-31: Warning (gpio_property): /bus@40000000/motherboard-bus@40000000/leds/user6:gpio: cell 0 is not
a phandle reference
dts/vexpress-modified.dts:374.6-31: Warning (gpio_property): /bus@40000000/motherboard-bus@40000000/leds/user7:gpio: cell 0 is not
a phandle reference
dts/vexpress-modified.dts:380.6-31: Warning (gpio_property): /bus@40000000/motherboard-bus@40000000/leds/user8:gpio: cell 0 is not
a phandle reference
DTB derlendi OK
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$
```

7) AŞAMA 6 — CHARACTER DRIVER ANALİZİ

7.1 Kullanılan Prompt

Aşama 6 için aşağıdaki prompt Claude Code'a verildi:

"Görev: Örnek bir Linux character driver yaz, analiz et ve rapor için belgele. Driver şu özelliklere sahip olmalı: module_init ve module_exit, struct file_operations içinde open, read, write, release fonksiyonları. open 'Driver açıldı' mesajını printk ile yazsın, read kullanıcıya sahte sıcaklık verisi döndürsün, write kullanıcıdan gelen veriyi printk ile loglasın, release 'Driver kapatıldı' printk yazsın. Major number alloc_chrdev_region ile dinamik alınsın, class_create ve device_create ile /dev/temp_sensor oluşturulsun."

Bu prompt tercih edildi çünkü character driver'ın tüm temel bileşenlerini tek seferde kapsamaktadır. Hem çalışan kod üretilmesi hem de kod analizi belgesi hazırlanması aynı anda talep edildi.

7.2 file_operations Açıklaması

file_operations, kullanıcı alanından gelen sistem çağrılarını (open, read, write, close) kernel driver'ındaki gerçek fonksiyonlara bağlayan bir fonksiyon tablosudur. VFS (Virtual File System) ile driver arasındaki köprü görevini üstlenir.

temp_driver.c içindeki file_operations tanımı şu şekildedir:

```
static const struct file_operations temp_fops = { .owner = THIS_MODULE, .open = temp_open, .read = temp_read, .write = temp_write, .release = temp_release };
```

Kullanıcı /dev/temp_sensor'ı açtığında çağrı zinciri şöyle işler:

open("/dev/temp_sensor") → VFS → file_operations.open → temp_open()

7.3 Kernel Modül Yaşam Döngüsü

Yükleme (insmod temp_driver.ko):

insmod komutu çalıştırılınca module_init() tetiklenir. alloc_chrdev_region() ile major ve minor numara dinamik olarak alınır. cdev_init() ve cdev_add() ile driver çekirdeğe kaydedilir. class_create() ile /sys/class/temp/ dizini oluşturulur. device_create() ile /dev/temp_sensor dosyası oluşturulur.

Kullanıcı Erişimi:

open("/dev/temp_sensor") çağrısı file_operations.open üzerinden temp_open() fonksiyonunu tetikler. read(fd, buf, n) çağrısı temp_read() fonksiyonunu tetikler ve kullanıcıya sıcaklık verisi döner. write(fd, buf, n) çağrısı temp_write() fonksiyonunu tetikler ve veri kernel loguna yazılır. close(fd) çağrısı temp_release() fonksiyonunu tetikler.

Kaldırma (rmmod temp_driver):

rmmod komutu çalıştırılınca module_exit() tetiklenir. device_destroy() ile /dev/temp_sensor silinir. class_destroy() ile /sys/class/temp/ dizini silinir. cdev_del() ile çekirdek kaydı silinir. unregister_chrdev_region() ile major ve minor numara serbest bırakılır.

7.4 open/read/write Akışı

open() — Kullanıcı /dev/temp_sensor'ı açtığında çağrılır. Bu driver'da doğrulama gerekmediğinden yalnızca printk ile "Driver acildi" logu bırakarak 0 döndürür.

read() — Kullanıcı read() yaptığında "25.3 C\n" sabit dizisini copy_to_user() ile kullanıcı belleğine güvenli biçimde kopyalar. copy_to_user() kullanılmasının nedeni kernel belleğine doğrudan erişimin güvenli olmamasıdır. loff_t offset takibi sayesinde ikinci çağrıda 0 döndürerek EOF davranışı simüle edilir, böylece cat /dev/temp_sensor sonsuz döngüye girmez.

write() — Kullanıcı write() veya echo "test" > /dev/temp_sensor yaptığında copy_from_user() ile kullanıcı verisini 64 baytlık çekirdek tamponuna çeker ve printk ile çekirdek loguna (dmesg) yazar. copy_from_user() kullanılmasının nedeni kullanıcı alanı belleğine doğrudan erişimin güvenli olmamasıdır.

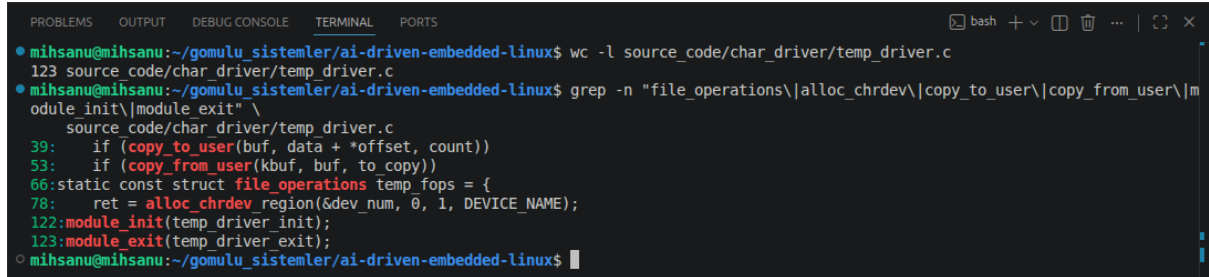
release() — Kullanıcı close() yaptığında çağrılır. Kaynak serbest bırakma gerekmediğinden yalnızca printk ile "Driver kapatıldı" logu bırakır ve 0 döner.

7.4 Hata Ayıklama

Bu aşamada karşılaşılan doğrudan bir derleme veya çalışma hatası olmadı. Ancak şu önemli nokta rapor için belirtilmelidir: Driver yalnızca host Linux üzerinde derlenebilir. QEMU

içindeki minimal rootfs'te kernel header dosyaları ve derleme araçları bulunmadığından driver QEMU içinde derlenemez. Bu durum gerçek gömülü sistemlerde de geçerlidir; driver her zaman host üzerinde cross-compile edilerek hedefe aktarılır.

7.6 Ekran Görüntüsü



```
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ wc -l source_code/char_driver/temp_driver.c
123 source_code/char_driver/temp_driver.c
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$ grep -n "file_operations\\|alloc_chrdev\\|copy_to_user\\|copy_from_user\\|module_init\\|module_exit" \
source_code/char_driver/temp_driver.c
39:     if (copy_to_user(buf, data + *offset, count))
53:     if (copy_from_user(kbuf, buf, to_copy))
66: static const struct file_operations temp_fops = {
78:     ret = alloc_chrdev_region(&dev_num, 0, 1, DEVICE_NAME);
122: module_init(temp_driver_init);
123: module_exit(temp_driver_exit);
mihsanu@mihsanu:~/gomulu_sistemler/ai-driven-embedded-linux$
```

8) SONUÇ

Bu projede yapay zekâ destekli bir geliştirme süreci izlenerek uçtan uca çalışan bir ARM Embedded Linux sistemi başarıyla oluşturuldu. Altı aşamanın tamamı tamamlandı ve her aşama QEMU ortamında doğrulandı.

AI Kullanımının Değerlendirmesi

Projede Claude AI kullanımı "kod kopyalama" değil "mühendislik sürecini hızlandırma" amacıyla uygulandı. Her üretilen kod parçası analiz edildi, eksiklikler tespit edildi ve gerektiğinde düzeltmeler yapıldı. AI'nın ürettiği kodun neden çalıştığını anlamak, çalışmasından daha önemli bir kazanım olarak öne çıktı.

Prompt mühendisliği sürecinde tüm proje bağlamını tek bir prompt ile vermek, AI'nın tutarlı ve birbirine entegre çözümler üretmesini sağladı. Her aşamada önceki bağlam korunarak ilerlendi.

Karşılaşılan ve Çözülen Başlıca Sorunlar

Sorun 1 — ENOEXEC hatası: initramfs içine yanlışlıkla x86-64 formatında BusyBox binary'si eklendi. ARM kernel bu binary'yi çalıştıramadı. file komutu ile mimari uyumsuzluğu tespit edildi, Debian armhf reposundan ARM binary'si indirilerek çözüldü.

Sorun 2 — tc.c derleme hatası: BusyBox kaynak koddan derlenirken tc.c dosyasında hata alındı. make menuconfig ile tc komutu devre dışı bırakılarak çözüldü.

Sorun 3 — Konsol blokajı: QEMU başlatma parametresindeki rdinit değeri yanlış binary'ye işaret ediyordu. Parametre /sbin/init olarak düzeltilerek BusyBox init'in doğru çalışması sağlandı.

Kazanılan Beceriler

Cross-compilation ve ABI kavramı: Bir platformda başka bir platform için kod derlemenin temel mantığı ve -static parametresinin gömülü sistemlerde neden zorunlu olduğu öğrenildi.

initramfs yapısı ve Linux boot süreci: Kernel'in açılış aşamalarında initramfs'in rolü, cpio formatı ve BusyBox ile minimal sistem kurulumu deneyimlendi.

IPC — fork ve pipe: Producer-consumer modelinde iki process arasında tek yönlü veri akışı, EOF yönetimi ve waitpid ile senkronizasyon uygulamalı olarak öğrenildi.

Device Tree tersine mühendisliği: Mevcut bir DTB'nin decompile edilerek incelenmesi, uygun konuma node eklenmesi ve /proc/device-tree ile doğrulanması deneyimlendi.

Character driver ve VFS katmanı: file_operations yapısının kernel ile kullanıcı alanı arasındaki köprü rolü, copy_to_user ve copy_from_user fonksiyonlarının güvenli bellek transferindeki önemi anlaşıldı.

9) KAYNAKLAR

1. Linux Kernel Documentation — <https://kernel.org/doc>
2. BusyBox Resmi Sitesi — <https://busybox.net>
3. QEMU Documentation — <https://qemu.org/docs>
4. Debian ARM Packages — <https://packages.debian.org>
5. Device Tree Specification — <https://devicetree.org/specifications>
6. Marmara Üniversitesi Gömülü Sistemler Ders Notları, Modül 1-10, Dr. Sebahattin Babur